

Abstract	2
Introduction	2
System Specifications	2
Device design	4
Symbol Diagram	4
VGA Specification	4
Block Diagrams	5
State Diagram	5
Process Diagram	6
Results	6
Timing Diagrams and Screen Capture	6
Brief Explanation of Results	7
Conclusion	7
References	7
Appendix	7

Abstract

This project presents the design and implementation of a Simple Video Game Processor on a Xilinx Spartan-3E FPGA, generating real-time VGA graphics at 640x480 and 60 Hz using a 25 MHz pixel clock. The system renders a green play field with white borders, two player-controlled paddles, and a moving ball that reacts to collisions and goal events, all implemented purely in VHDL through hardware-based timing, synchronization, and color-generation logic. User interactions are handled through onboard switches, enabling responsive paddle movement. The results confirm full compliance with VGA timing requirements: waveform analysis shows correct HSync and VSync pulses, proper hpos/vpos counter operation, and accurate active display intervals. Screen captures verify stable rendering, smooth object motion, and correct game behavior, including ball reflection and scoring transitions. Overall, the project successfully demonstrates real-time digital system design, hardware control, and FPGA-based video generation.

Introduction

This project focuses on the design and implementation of a Simple Video Game Processor (SVGP) using the Xilinx Spartan-3E FPGA development board. The main goal is to create a real-time video output system compatible with the VGA standard, allowing visual display and user interaction through hardware controls. The SVGP generates a static game field in green with white borders and includes dynamic elements such as a yellow ball and two player paddles in blue and pink. The paddles are controlled by onboard switches, while the ball moves automatically, reflecting off the borders and paddles according to collision logic. Moreover, the system is designed using VHDL and implemented on the FPGA, which manages video timing, synchronization signals (HSync and VSync), and pixel color generation. When the ball passes through the goal area, it briefly changes color to red before resetting to the center, demonstrating event-driven behavior in real time. This project provides valuable experience in digital design, hardware interfacing, and VGA signal generation, reinforcing theoretical concepts of real-time processing and modular FPGA design in practical applications.

System Specifications

The Simple Video Game Processor (SVGP) is implemented on the Xilinx Spartan-3E FPGA, producing a 640×480 VGA display at 60 Hz using a 25 MHz pixel clock. The system includes a static green field with white borders, two player paddles (blue and pink) controlled by onboard switches, and a yellow ball that moves automatically, reflecting off borders and paddles based on collision logic. When the ball enters a goal, it turns red and resets to the center. The SVGP and VGA controller will use a 25 MHz DAC clock derived from the 50 MHz core clock, 8-bit RGB outputs for pixel color data, and HSync/VSync signals for synchronization. The system meets three main requirements: displaying a static background, enabling vertical paddle movement, and controlling a dynamically bouncing ball with proper reset behavior.

Signal Name	Direction	Description	Mapped FPGA Pin
clk	Input	50 MHz board clock used to generate 25 MHz pixel clock	c9
DAC_CLK	Output	25 MHz VGA pixel clock for DAC	a4
H	Output	Horizontal synchronization signal (HSync)	c5
V	Output	Vertical synchronization signal (VSync)	d5
Rout[7:0]	Output	8-bit red color channel	b16, a16, d14, c14, b14, a14, b13, a13
Gout[7:0]	Output	8-bit green color channel	f9, e9, d11, c11, f11, e11, e12, f12
Bout[7:0]	Output	8-bit blue color channel	a6, b6, e7, f7, d7, c7, f8, e8
SW0	Input	Player control switch 0	N17
SW1	Input	Player control switch 1	H18
SW2	Input	Player control switch 2	L14
SW3	Input	Player control switch 3	L13

Table 1: Port Mapping

Table 1 shows the port mapping between the Spartan-3E FPGA board and the VHDL design, defining how hardware signals are connected to the VGA interface and control inputs. The FPGA's 50 MHz clock is used to generate the 25 MHz pixel clock required for VGA timing, while synchronization signals (HSync and VSync) ensure proper image rendering on the display. The RGB output ports deliver 8-bit color data to the VGA DAC, producing the visible game graphics. Additionally, the onboard switches (SW0-SW3) serve as user inputs to control paddle movement, enabling real-time interaction with the game.

Device design

Symbol Diagram

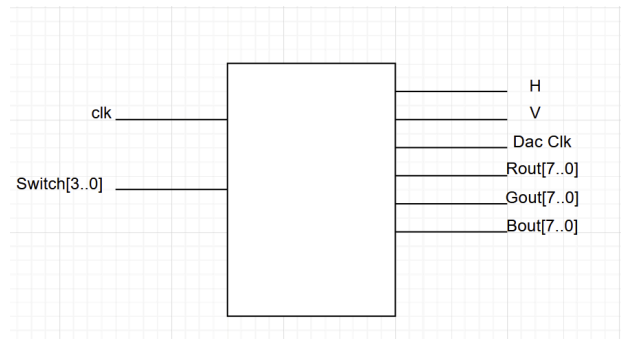


Figure 1: Symbol Diagram

Figure 1, the symbol diagram represents the top-level design of the Simple Video Game Processor (SVGP) implemented on a Xilinx Spartan-3E FPGA. The circuit generates VGA video signals (H, V, Rout, Gout, Bout, Dac Clk) based on user inputs from switches and a clock source. It controls the display of static and dynamic elements, such as the game field, players, and moving ball, on a VGA monitor in real time. This design demonstrates FPGA-based video signal generation and I/O interfacing techniques using VHDL.

VGA Specification

Parameter	Clock Cycles
Complete Line	800
Front Porch	16
Sync Pulse	96
Back Porch	48
Active image area	640

Table 1: VGA Horizontal Parameters

Parameter	Clock Cycles
Complete Frame	525
Front Porch	10
Sync Pulse	2
Back Porch	33
Active image area	480

Table 2: VGA Vertical Parameters

Figure 2: VGA Specification

Figure 2, the VGA specifications used in this project define the timing and synchronization parameters required to generate a standard 640×480 display resolution at a 60 Hz refresh rate. The system operates with a 25 MHz pixel clock, which determines the timing for each horizontal line. As shown in Table I, one complete horizontal line consists of 800 clock cycles, divided into distinct intervals: a front porch of 16 cycles, a sync pulse of 96 cycles, a back porch of 48 cycles, and an active image area of 640 cycles, during which visible pixels are displayed. Similarly, the vertical timing parameters, summarized in Table 2, define a total of 525 lines per

frame. This includes a front porch of 10 lines, a sync pulse of 2 lines, a back porch of 33 lines, and an active display area of 480 lines. Together, these parameters ensure that the monitor receives properly timed horizontal (HSync) and vertical (VSync) synchronization signals, maintaining a stable and flicker-free display. This configuration adheres to the VGA standard and provides a reliable foundation for displaying the game's graphical output.

Block Diagrams

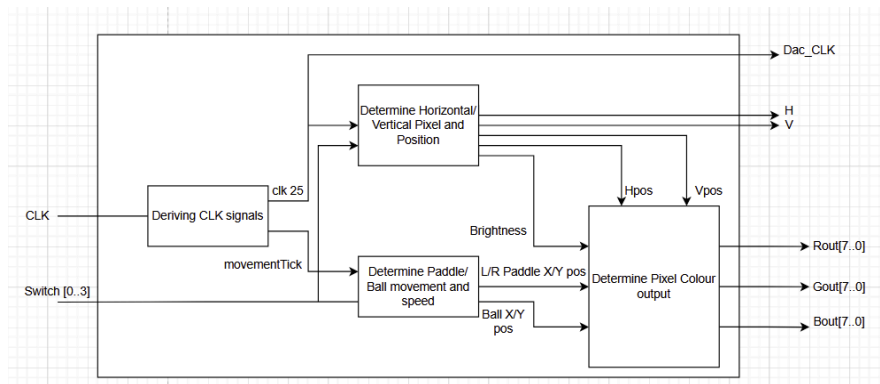


Figure 3: Block Diagram

Figure 3, the block diagram shows the internal architecture of the Simple Video Game Processor (SVGP) implemented on the FPGA. The system uses a master clock to derive timing signals required for VGA synchronization and game movement control. The horizontal and vertical position trackers generate pixel coordinates, while the paddle and ball control unit determines object movement and speed based on switch inputs. The pixel color output block combines these signals to generate the RGB outputs, producing the game display on the VGA monitor in real time.

State Diagram

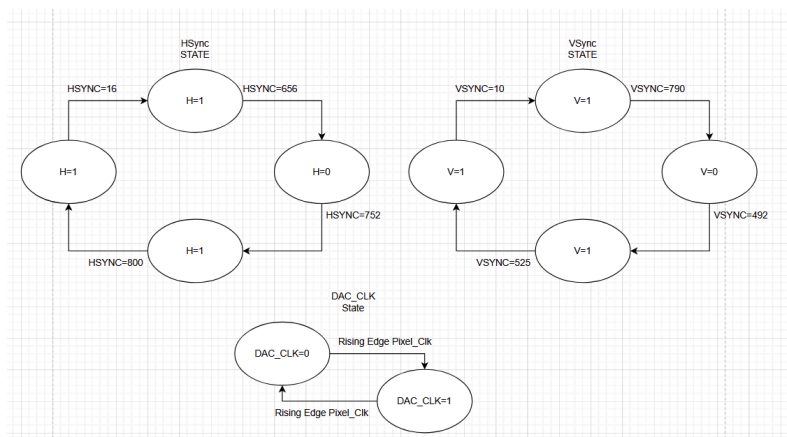


Figure 4: State Diagram

Figure 4, shows the state diagram, which represents the VGA synchronization process that controls horizontal (HSync) and vertical (VSync) timing for a 640×480 display at 60 Hz. The HSync state transitions occur at defined pixel counts (16, 656, 752, 800) to generate front porch, sync pulse, back porch, and active display periods. Similarly, the VSync state transitions at specific line counts (10, 490, 525, 790) to regulate frame timing. The DAC clock toggles with each rising edge of the pixel clock, ensuring synchronized pixel data transfer to the VGA output.

Process Diagram

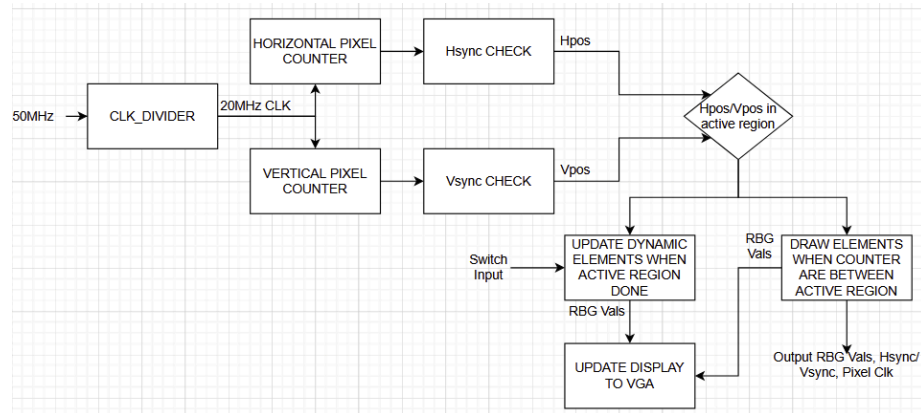
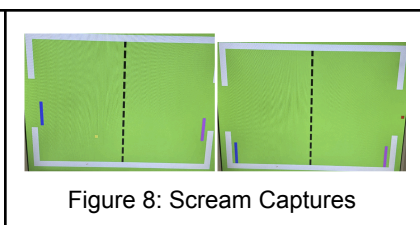
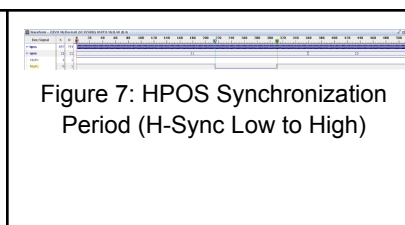
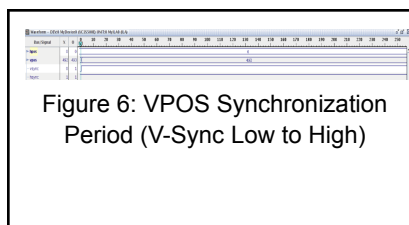


Figure 5: Process Diagram

Figure 5, the process diagram shows the overall VGA display control flow. The 50 MHz input clock is divided down to a 25 MHz pixel clock, which drives both the horizontal and vertical pixel counters. These counters generate timing positions (Hpos and Vpos) used by the HSync and VSync check modules to control synchronization signals. When the counters fall within the active display region, pixel data (RGB values) are drawn to the screen. Outside the active region, the system updates dynamic elements, such as paddle and ball positions, based on switch inputs, and then outputs the updated RGB values, synchronization signals, and pixel clock to the VGA display.

Results

Timing Diagrams and Screen Capture



Brief Explanation of Results

Figure 6, shows the waveform of the VGA timing signals generated by the FPGA, including the horizontal position (hpos), vertical position (vpos), and the HSync/VSyn pulses. The counters correctly increment through the active display region before resetting at the end of each line and frame. HSync and VSync toggle at the expected intervals, confirming that the VGA synchronization logic follows the 640x480 at 60 Hz specification. This validates that the system produces stable and properly timed video signals for the display. Figure 7, shows the waveform of hpos counting horizontally across the active display region, with vpos remaining stable during this line. hsync is low only for a short pulse at the left, marking the horizontal sync period, while vsync stays high, indicating the system is not in a vertical sync interval. The highlighted region marks the active video window where pixel data would be valid. Therefore, the waveforms confirm that the hpos and vpos counters sweep through the correct active ranges before resetting, matching the required 640-pixel by 480-line resolution. Figure 8, shows the real-time VGA screen output of the Simple Video Game Processor running on the FPGA. The first image displays gameplay with the green field, white borders, paddles, and the yellow moving ball. The second image shows a goal event, where the ball turns red as it crosses the boundary before resetting to the center. This shows correct VGA rendering, paddle control, ball movement, and goal-detection behavior.

Conclusion

The Simple Video Game Processor (SVGP) project successfully implemented a real-time VGA-based video game on the Xilinx Spartan-3E FPGA. The system displayed a stable 640×480 game field, controlled moving objects, and user interactions through hardware switches. It demonstrated effective VGA synchronization, collision detection, and motion control using VHDL. Overall, the project met its objectives and provided valuable experience in FPGA design, real-time processing, and digital system integration.

References

[1] COE758 Digital System Engineering, *Project #2 – Simple Video Game Processor for VGA*, Toronto Metropolitan University, 2025. [Online]. Available: <https://courses.torontomu.ca/d2l/le/content/1072788/viewContent/6603170/View>. [Accessed: Nov. 19, 2025].

Appendix

Game VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```

use IEEE.NUMERIC_STD.ALL;

entity Game is
  Port (
    clk      : in  STD_LOGIC;
    H        : out STD_LOGIC;
    V        : out STD_LOGIC;
    DAC_CLK  : out STD_LOGIC;
    Bout     : out STD_LOGIC_VECTOR (7 downto 0);
    Gout     : out STD_LOGIC_VECTOR (7 downto 0);
    Rout     : out STD_LOGIC_VECTOR (7 downto 0);
    SW0      : in  STD_LOGIC;
    SW1      : in  STD_LOGIC;
    SW2      : in  STD_LOGIC;
    SW3      : in  STD_LOGIC
  );
end Game;

architecture Behavioral of Game is

  signal hcounter, vcounter : integer range 0 to 799;
  signal pixel_clk, refresh_clk : std_logic;
  signal refresh_cntr : integer := 0;

  signal R, G, B : std_logic_vector(7 downto 0);

  signal top_border_x1, top_border_x2 : integer := 10;
  signal top_border_y1, top_border_y2 : integer := 10;
  signal top_border_x3, top_border_x4 : integer := 10;
  signal top_border_y3, top_border_y4 : integer := 20;
  signal top_border_x5, top_border_x6 : integer := 620;
  signal top_border_y5, top_border_y6 : integer := 10;

  signal b_border_x1, b_border_x2 : integer := 10;
  signal b_border_y1, b_border_y2 : integer := 300;
  signal b_border_x3, b_border_x4 : integer := 10;
  signal b_border_y3, b_border_y4 : integer := 470;
  signal b_border_x5, b_border_x6 : integer := 620;
  signal b_border_y5, b_border_y6 : integer := 300;

  signal m_line_x1, m_line_x2 : integer := 318;
  signal m_line_y1, m_line_y2 : integer := 30;

  signal c_border_x1, c_border_x2 : integer := 300;
  signal c_border_y1, c_border_y2 : integer := 220;
  signal cc_border_x1, cc_border_x2 : integer := 305;
  signal cc_border_y1, cc_border_y2 : integer := 225;

  signal r_paddle_x1, r_paddle_x2 : integer := 600;
  signal r_paddle_y1, r_paddle_y2 : integer := 200;
  signal b_paddle_x1, b_paddle_x2 : integer := 25;
  signal b_paddle_y1, b_paddle_y2 : integer := 200;

  signal ball_x1, ball_x2 : integer := 310;
  signal ball_y1, ball_y2 : integer := 230;

  signal r_goal_x, b_goal_x : integer := 625;
  signal score, reset : std_logic;
  signal ball_x_inc, ball_y_inc : std_logic;

begin

  process (clk)
  begin
    if rising_edge(clk) then
      pixel_clk <= not pixel_clk;
    end if;
  end process;

```



```

DAC_CLK <= pixel_clk;

process (pixel_clk)
begin
    if rising_edge(pixel_clk) then
        hcounter <= hcounter + 1;
        if (hcounter = 799) then
            hcounter <= 0;
            vcounter <= vcounter + 1;
        end if;
        if (vcounter = 524) then
            vcounter <= 0;
        end if;
    end if;
end process;

H <= '0' when hcounter <= 96 else '1';
V <= '0' when vcounter <= 2 else '1';

process
begin
    if (hcounter >= 143 and hcounter <= 783 and vcounter >= 34 and vcounter <= 514) then
        Rout <= R;
        Gout <= G;
        Bout <= B;
    else
        Rout <= (others => '0');
        Gout <= (others => '0');
        Bout <= (others => '0');
    end if;
end process;

process(pixel_clk)
begin
    if rising_edge(pixel_clk) then
        if (refresh_cntr >= 416667) then
            refresh_clk <= not refresh_clk;
            refresh_cntr <= 0;
        else
            refresh_cntr <= refresh_cntr + 1;
        end if;
    end if;
end process;

process(refresh_clk)
begin
    if rising_edge(refresh_clk) then

        if (ball_x1 <= top_border_x2 + 7 and (ball_y1 >= top_border_y4 and ball_y2 <= top_border_y6 + 2)) then
            ball_x_inc <= '1';
        elsif (ball_x1 <= b_border_x2 + 7 and (ball_y1 >= b_border_y5 - 2 and ball_y2 <= b_border_y6)) then
            ball_x_inc <= '1';
        elsif (ball_x2 >= top_border_x5 - 7 and (ball_y2 >= top_border_y4 and ball_y1 <= top_border_y6)) then
            ball_x_inc <= '0';
        elsif (ball_x2 >= b_border_x5 - 7 and (ball_y2 >= b_border_y1 and ball_y1 <= b_border_y6)) then
            ball_x_inc <= '0';
        end if;

        if (ball_x1 <= b_paddle_x2 and (ball_y1 >= b_paddle_y1 and ball_y2 <= b_paddle_y2)) then
            ball_x_inc <= '1';
        elsif (ball_x2 <= b_paddle_x1 + 7 and (ball_y2 >= b_paddle_y2 - 7 and ball_y1 <= b_paddle_y1 + 7)) then
            ball_x_inc <= '0';
        elsif (ball_x1 >= r_paddle_x2 - 7 and (ball_y2 >= -7 and ball_y1 <= r_paddle_y1 + 7)) then
            ball_x_inc <= '1';
        end if;

        if (ball_y1 <= top_border_y4 + 7) then
            ball_y_inc <= '0';
        elsif (ball_y2 >= b_border_y3 - 7) then

```

```

        ball_y_inc <= '1';
    end if;

    if (ball_x1 < b_goal_x or ball_x2 > r_goal_x) then
        score <= '1';
    else
        score <= '0';
    end if;

    if (ball_x1 <= 5 or ball_x2 >= 635) then
        ball_x1 <= 310;
        ball_x2 <= 325;
        ball_y1 <= 230;
        ball_y2 <= 245;
        ball_x_inc <= '1';
        ball_y_inc <= '1';
    else
        if (ball_x_inc = '1') then
            ball_x1 <= ball_x1 + 6;
            ball_x2 <= ball_x2 + 6;
        else
            ball_x1 <= ball_x1 - 6;
            ball_x2 <= ball_x2 - 6;
        end if;

        if (ball_y_inc = '1') then
            ball_y1 <= ball_y1 - 6;
            ball_y2 <= ball_y2 - 6;
        else
            ball_y1 <= ball_y1 + 6;
            ball_y2 <= ball_y2 + 6;
        end if;
    end if;
end if;
end process;

process(refresh_clk)
begin
    if rising_edge(refresh_clk) then
        if (SW2 = '1' and r_paddle_y1 > top_border_y4) then
            r_paddle_y1 <= r_paddle_y1 - 10;
            r_paddle_y2 <= r_paddle_y2 - 10;
        elsif (SW2 = '0' and r_paddle_y2 < b_border_y3) then
            r_paddle_y1 <= r_paddle_y1 + 10;
            r_paddle_y2 <= r_paddle_y2 + 10;
        end if;

        if (SW0 = '1' and b_paddle_y1 > top_border_y4) then
            b_paddle_y1 <= b_paddle_y1 - 10;
            b_paddle_y2 <= b_paddle_y2 - 10;
        elsif (SW0 = '0' and b_paddle_y2 < b_border_y3) then
            b_paddle_y1 <= b_paddle_y1 + 10;
            b_paddle_y2 <= b_paddle_y2 + 10;
        end if;
    end if;
end process;

process(hcounter, vcounter)
    variable x, y : integer range 0 to 639;
begin
    x := hcounter - 143;
    y := vcounter - 34;

    R <= "00000000";
    G <= "11111111";
    B <= "00000000";

    if (x > ball_x1 and x < ball_x2 and y > ball_y1 and y < ball_y2) then
        if (score = '1') then

```

```

    R <= "11111111"; G <= "00000000"; B <= "00000000";
else
    R <= "11111111"; G <= "11111111"; B <= "00000000";
end if;

elsif (x > top_border_x1 and x < top_border_x2 and y > top_border_y1 and y < top_border_y2) or
(x > top_border_x3 and x < top_border_x4 and y > top_border_y3 and y < top_border_y4) or
(x > top_border_x5 and x < top_border_x6 and y > top_border_y5 and y < top_border_y6) or
(x > b_border_x1 and x < b_border_x2 and y > b_border_y1 and y < b_border_y2) or
(x > b_border_x3 and x < b_border_x4 and y > b_border_y3 and y < b_border_y4) or
(x > b_border_x5 and x < b_border_x6 and y > b_border_y5 and y < b_border_y6) then
    R <= "00000000"; G <= "00000000"; B <= "00000000";

elsif (x > cc_border_x1 and x < cc_border_x2 and y > cc_border_y1 and y < cc_border_y2) then
    R <= "00000000"; G <= "11111111"; B <= "00000000";

elsif (x > m_line_x1 and x < m_line_x2 and y > m_line_y1 and y < m_line_y2) or
(x > c_border_x1 and x < c_border_x2 and y > c_border_y1 and y < c_border_y2) then
    R <= "11111111"; G <= "11111111"; B <= "11111111";

elsif (x > r_paddle_x1 and x < r_paddle_x2 and y > r_paddle_y1 and y < r_paddle_y2) then
    R <= "11111111"; G <= "00000000"; B <= "00000000";

elsif (x > b_paddle_x1 and x < b_paddle_x2 and y > b_paddle_y1 and y < b_paddle_y2) then
    R <= "00000000"; G <= "00000000"; B <= "11111111";
end if;
end process;
end Behavioral;

```